

CICD勉強会

- まえがき

プロジェクトなどでほとんど触れることのないCICDパイプラインの実装や処理内容を理解するための勉強会。

BookStation（Webアプリ）をAzureでデプロイする際のCICDパイプラインを実装します。（コードフォーマット確認と単体試験の実装）

※新規作成からWebアプリケーションのデプロイまでの最低限の作成方法は、別の学習記事で記載済みなので、そちらを参照してください。（[CICD構成の作り方](#)）

成果物

今回作成した成果物は以下の通りです。

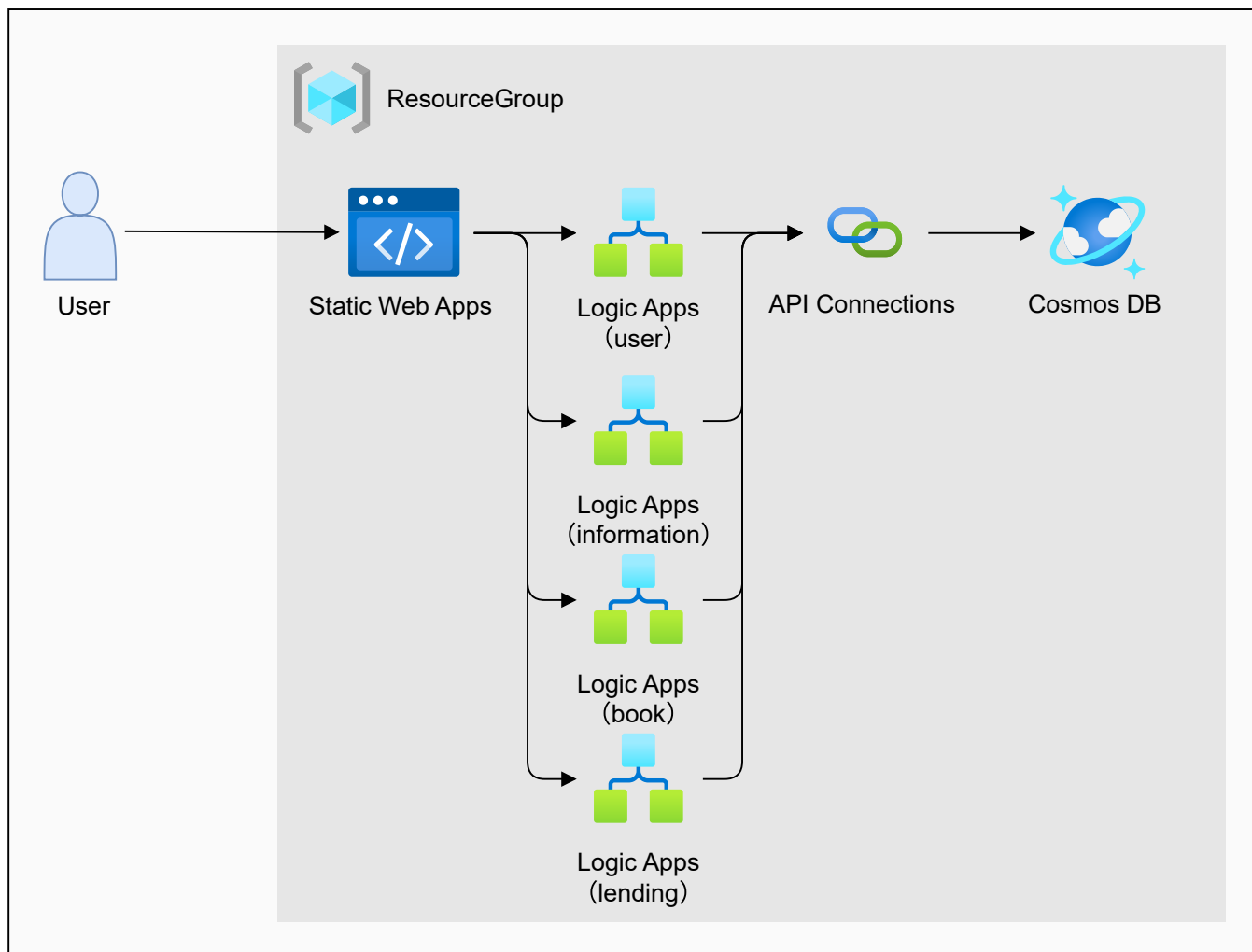
CICDの実装や仕組み自体はyamlにすべて記載されています。

```
AzureStaticWebApps-BookStation/
├─ github/
│   └─ workflows/
│       └─ azure-static-web-apps-mango-river-03a6bc500.yml    # CICDを構成する設定ファイル
├─ node_modules
├─ public                                                       # Webアプリの実装部分
├─ src                                                         # Webアプリの実装部分
└─ :
```

また、本学習と関連はありませんがWebアプリの構成は以下の通りです。

ローカル環境に資材を配置せず、クラウド上で完結させています。

[WebアプリケーションURL](#)



CICD実装手順

今回作成したジョブは、最も基本的なジョブである「コードフォーマットチェック」と「単体試験」になります。それぞれ章ごとに作り方を記載しています。

コードフォーマット確認

コードフォーマット確認ジョブでの最終的なymlは以下の通りです。

/AzureStaticWebApps-BookStation/.github/workflows/azure-static-web-apps-mango-river-03a6bc500.yml

```

jobs:
  check_codeFormatting_job: # ジョブ名 (yml内での定義名)
    runs-on: ubuntu-latest # ジョブの実行環境の指定 (Linuxを指定)
    name: Check Code Formatting Job # ジョブ名 (GitHubで確認可能)
    steps:
      - uses: actions/checkout@v3 # デプロイ対象のリポジトリのチェックアウトを実施
      - name: Install Node.js # 次ステップ名 (GitHubで確認可能)
  
```

```

    uses: actions/setup-node@v3 # ジョブの実行環境に
Node.jsをインストール
    with:
      node-version: '16' # インストールする
Node.jsのバージョンを指定
      - name: Install Prettier # 次ステップ名 (GitHub
で確認可能)
        run: npm install --global prettier # コードフォーマッター
        (prettier) をインストール
      - name: Check Code Formatting with Prettier # 次ステップ名 (GitHub
で確認可能)
        run: prettier --check "**/*.{js,json,css,scss}" # ソースコードがフォーマ
ットされているか確認 (修正はしない)

```

各項目の解説

- `runs-on: ubuntu-latest`
本ジョブを実行する環境の指定をします。様々選べますが、今回はLinux (ubuntu) を選択します。
- `uses: actions/checkout@v3`
実行環境に対してデプロイ対象のリポジトリをチェックアウトします。
※直前の処理で生成したばかりの環境なので、まだ空の状態なのでソースコードやモジュールを準備する必要があります。
- `uses: actions/setup-node@v3`
実行環境に対してNodejsをインストールします。
※こちらも同様に、ジョブで処理をさせるための準備作業になります。
- `with: node-version: '16'`
Nodejsのバージョン指定になります。インストールアクションの直後に`with`を記載してアクションに引数 (バージョン) を渡します。
- `run: npm install --global prettier`
実行環境にコードフォーマッター (prettier) をインストールします。この後のコマンドでprettierを使用します。
- `run: prettier --check "**/*.{js,json,css,scss}"`
対象のソースに対して、コードフォーマットに順守しているかどうか確認します。
※`prettier --check` はファイルを書き換えずに確認のみ行います。違反があった場合はジョブが失敗し、後続の `build_and_deploy_job` はブロックされます。

※@3などの記載はGitHubが用意した公式のアクションになります。

[checkoutアクションのGithubリポジトリ](#)

実際にデプロイを実施した時の表示は以下のようになります。

フォーマットに違反があった場合には、それらがわかるようになっています。

Check Code Formatting

failed 6 hours ago in 10s

>  Set up job

>  Run actions/checkout@v3

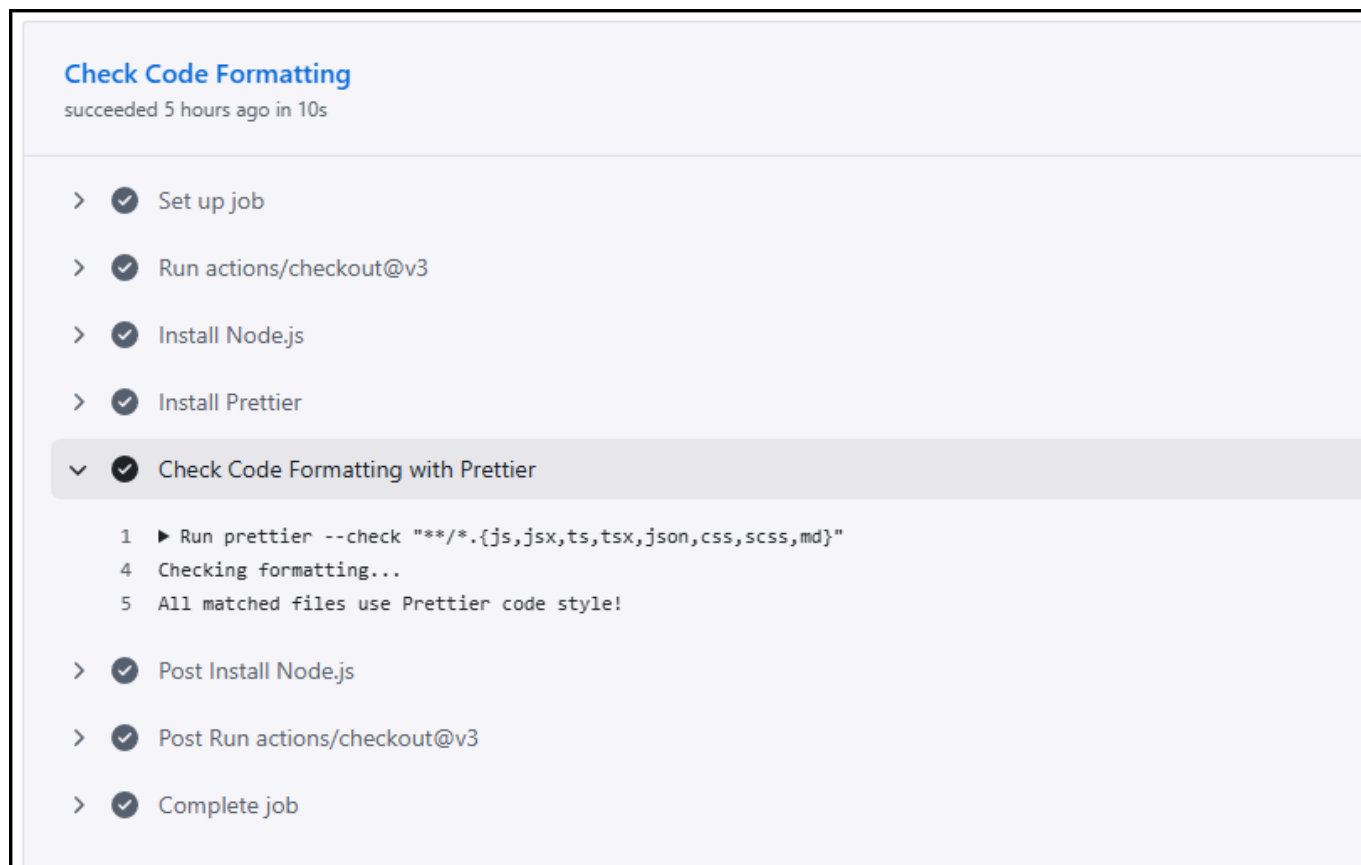
>  Install Node.js

>  Install Prettier

>  Check Code Formatting with Prettier

```
1 ▶ Run prettier --check "**/*.{js,jsx,ts,tsx,json,css,scss,md}"
4 Checking formatting...
5 [warn] .vscode/extensions.json
6 [warn] .vscode/launch.json
7 [warn] .vscode/settings.json
8 [warn] .vscode/tasks.json
9 [warn] api/host.json
10 [warn] api/src/functions/dataaccess.js
11 [warn] api/src/functions/message.js
12 [warn] api/src/index.js
13 [warn] babel.config.js
14 [warn] public/css/common.css
15 [warn] public/css/sb-admin.min.css
16 [warn] public/scss/_global.scss
17 [warn] public/scss/_navbar.scss
18 [warn] README.md
19 [warn] src/main.js
20 [warn] src/Utils/AjaxUtil.js
21 [warn] src/Utils/const/UserConst.js
22 [warn] src/Utils/sb-admin.js
23 [warn] src/Utils/UserUtil.js
24 [warn] staticwebapp.config.json
25 [warn] staticwebapp.database.config.json
26 [warn] Code style issues found in 21 files. Run Prettier with --write to fix.
27 Error: Process completed with exit code 1.
```

フォーマット修正後のジョブは以下の通りです。きちんとジョブが成功していることがわかります。



単体試験

単体試験ジョブでの最終的なymlは以下の通りです。

/AzureStaticWebApps-BookStation/.github/workflows/azure-static-web-apps-mango-river-03a6bc500.yml

```
jobs:
  unit_test_job:
    runs-on: ubuntu-latest
    name: Unit Test Job
    steps:
      - uses: actions/checkout@v3
      - name: Install Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '16'
      - name: Install dependencies
        run: npm ci
      - name: Run unit tests
        run: npm test
```

ジョブ名 (yml内での定義名)
 # ジョブの実行環境の指定 (Linuxを指定)
 # ジョブ名 (GitHubで確認可能)
 # デプロイ対象のリポジトリのチェックアウトを実
 施
 # 次ステップ名 (GitHubで確認可能)
 # ジョブの実行環境にNode.jsをインストール
 # インストールするNode.jsのバージョンを指定
 # 次ステップ名 (GitHubで確認可能)
 # node_modulesをインストール
 # 次ステップ名 (GitHubで確認可能)
 # package.jsonに定義されたスクリプト「test」
 を実行

各項目の解説

- `runs-on: ubuntu-latest`
本ジョブを実行する環境の指定をします。様々選べますが、今回はLinux (ubuntu) を選択します。
- `uses: actions/checkout@v3`
実行環境に対してデプロイ対象のリポジトリをチェックアウトします。
※直前の処理で生成したばかりの環境なので、まだ空の状態なのでソースコードやモジュールを準備する必要があります。
- `uses: actions/setup-node@v3`
実行環境に対してNode.jsをインストールします。
※こちらも同様に、ジョブで処理をさせるための準備作業になります。
- `with: node-version: '16'`
Node.jsのバージョン指定になります。インストールアクションの直後に`with`を記載してアクションに引数（バージョン）を渡します。
- `npm ci`
`package-lock.json`に記載された**厳密なバージョン**で依存パッケージをインストールします。
※`npm install` と異なり、lockファイルと差異があるとエラーになるためCI環境向き。←勉強中。
- `npm test`
`package.json`に定義したスクリプトを実行します。定義内容は、`jest`を実行するというものになっています。

/AzureStaticWebApps-BookStation/package.json

```
{
  "scripts": {
    "start": "serve -s dist",
    "serve": "vue-cli-service serve",
    "build": "vue-cli-service build",
    "lint": "vue-cli-service lint",
    "test": "jest" // これを実行
  },
}
```

単体試験ジョブ実行結果は以下の通りです。テストの実施内容ごとにきちんとログが出力されていることがわかります。

Unit Test

succeeded 6 hours ago in 31s

✓ Run unit tests

```
7
8 PASS src/utlis/__tests__/AjaxUtil.test.js
9   AjaxUtil
10     共通: レスポンスが ok でない場合
11       ✓ 404 のとき Error をスロー (13 ms)
12       ✓ スローされたエラーに response.status が含まれる (1 ms)
13     共通: クエリパラメータ
14       ✓ params が null のとき URL に ? が付かない (1 ms)
15       ✓ undefined / null の値はクエリから除外される (1 ms)
16       ✓ 複数パラメータが正しく結合される (1 ms)
17     postSignIn
18       ✓ POST /api/signin にリクエストを送り data を返す (1 ms)
19     getUser
20       ✓ GET /api/users にクエリパラメータ付きでリクエスト (1 ms)
21     getUserByAuth
22       ✓ GET /api/users?auth=xxx でリクエスト (1 ms)
23     getUserById
24       ✓ GET /api/users/:id でリクエスト
25     postUser
26       ✓ POST /api/users にリクエスト
27     putUser
28       ✓ PUT /api/users にリクエスト (1 ms)
29     deleteUser
30       ✓ DELETE /api/users/:id でリクエスト
31     getAllSapBooks
32       ✓ GET /api/books でリクエスト (1 ms)
33     getAllSearchBooks
34       ✓ GET /api/books/search?word=xxx でリクエスト
35     getInformation
36       ✓ GET /api/information でリクエスト (1 ms)
37
38 Test Suites: 1 passed, 1 total
39 Tests:       15 passed, 15 total
40 Snapshots:   0 total
41 Time:        1.044 s
42 Ran all test suites.
```

付録（以下はデフォルトで生成されたreadmeの内容）

[Azure Static Web Apps](#) allows you to easily build [Vue.js](#) apps in minutes. Use this repo with the [Vue quickstart](#) to build and customize a new static site.

Project setup

```
npm install
```

Compiles and hot-reloads for development

```
npm run serve
```

Compiles and minifies for production

```
npm run build
```

Lints and fixes files

```
npm run lint
```

Customize configuration

See [Configuration Reference](#).